

UAE Food-Aggregator Data Pipeline

Talabat UAE — End-to-End Technical Documentation

Prepared for: Management and Technology Leadership

Scope: Complete pipeline, from target discovery through monthly deliverable

Status: Production, operating on a monthly cycle

Date: August 2026

1. Executive Summary

1.1 What This System Does

This pipeline builds and maintains a complete, structured dataset of the UAE restaurant market as represented on Talabat — the region's largest food delivery aggregator. It discovers every restaurant listing across the country, extracts each one's full menu with prices, enriches that raw data with standardised classifications and real-world business details from Google Maps, and delivers a clean, validated dataset to the Technology team each month.

The output powers a data-driven application for UAE clients: menu-level market intelligence, competitive price tracking, and month-over-month change detection across the entire national restaurant landscape.

1.2 Current Scale

Measure	Value
UAE delivery zones swept	646
Restaurants under management	22,327
Menu items captured and enriched	2,094,478
Distinct menu item names standardised	524,207
Restaurant logos collected	22,322
Ingredient records derived	2,176,348
Monthly deliverable size	620 MB

1.3 What Makes This Difficult

Talabat actively defends against automated access. Every stage of this pipeline exists because a naive approach fails:

Bot protection. Standard HTTP libraries are fingerprinted and blocked at the TLS layer before a request completes. The pipeline uses browser-impersonating clients and a rotating residential proxy network with UAE exit nodes.

Scale against rate limits. 22,327 restaurants and 2.1 million menu items cannot be fetched naively. Throughput was benchmarked to find the sustainable ceiling — measured, not guessed — and the pipeline runs at that rate with circuit breakers and automatic retry.

Unstructured, multilingual source data. UAE menus mix English and Arabic, carry emoji, and are written inconsistently by 22,000 different restaurants. The same product appears as chicken shawarma, shawarma sandwich and shawrma sandwich. Standardising this into a controlled vocabulary is the single largest engineering investment in the pipeline.

Incomplete business data. Talabat does not publish restaurant addresses, phone numbers, websites or coordinates in usable form. These are recovered from Google Maps via a separate enrichment pipeline.

1.4 Delivery Model

The system runs on a **monthly cycle**. Each month it re-scrapes every known restaurant to capture price and menu changes, discovers newly-listed restaurants, enriches and standardises everything, and ships a single

validated file to the Technology team. Every stage is scripted, resumable, and gated by automated validation before anything is released.

2. What We Deliver

2.1 The Monthly Deliverable

The Technology team receives one file per month:

talabat_unified_YYYYMM.jsonl	620 MB
------------------------------	--------

JSON Lines format — one complete, self-contained JSON object per line. This is a deliberate choice over a single JSON array: Tech can stream the file, split it across workers, or load it line by line without holding the entire dataset in memory. It also compresses to roughly 89 MB, making transfer straightforward.

2.2 Record Structure

Each line is one restaurant with its complete menu:

Field	Description
source_name	Platform identifier (talabat)
source_id	Talabat's own branch identifier — the primary join key
name	Restaurant name, normalised
cuisine	Primary cuisine
sub_cuisines, key_cuisines	Full cuisine arrays
restaurant_type	Full-Service / Quick-Service / Cafes / Bakery / Cloud Kitchen
outlet_type	Chain or Independent
chain_type	Local Chain / MNC Chain / null
chain_id	Stable brand identifier, shared by every branch of a brand
chain_locations_count	Number of branches for that brand
currency	AED
location	Structured address: raw, country, city, area, sublocality
geo	Latitude and longitude
contact_phone, website, maps_url	Business contact details
menu_items[]	Complete menu

Each menu item carries:

Field	Description
name	Item name, cleaned and title-cased
section	Menu category as the restaurant defined it
description	Item description
std_term	Standardised product term from a controlled vocabulary
price	Price in AED
ingredients[]	Derived ingredient list
is_popular	Popularity flag

2.3 The Derived Fields — Where the Value Is

source_id, name, section, description and price are captured directly from Talabat. They are the factual record.

std_term, ingredients, chain_id, restaurant_type, outlet_type and the structured location are **derived** — they do not exist on Talabat and are produced by this pipeline. They are what turn a scrape into a dataset:

- std_term makes 524,207 differently-written item names comparable. Without it, "how did chicken shawarma prices move this month" is unanswerable.
- chain_id groups every branch of a brand under one identifier, enabling brand-level analysis across thousands of outlets.
- ingredients supports dietary, allergen and supply-chain analysis.

2.4 Supporting Deliverables

Deliverable	Contents
Restaurant logos	One image per restaurant, organised by source_id
Ingredient reference	816 distinct ingredients with categories
Location reference	Area, district and city per restaurant
Change log (internal)	Month-over-month additions, price changes, delistings

3. System Overview

3.1 The Pipeline

Nine stages, each independently runnable, resumable, and validated:

1	TARGET DISCOVERY	646 UAE delivery zones mapped and swept
2	RESTAURANT IDENTIFICATION	↓ Confirm each URL is a real food business
3	CLASSIFICATION	↓ Cuisine, format, chain status, MNC status
4	MENU EXTRACTION	↓ Full menu, prices, descriptions per restaurant
5	STANDARDISATION	↓ std_term + ingredients via MyRA AI + mapping
6	BUSINESS ENRICHMENT	↓ Address, phone, website, coordinates, Maps URL
7	LOGO COLLECTION	↓ Restaurant logo images
8	MONTHLY REFRESH	↓ Re-scrape, delta computation, price velocity
9	ASSEMBLY & VALIDATION	↓ Unified deliverable + automated quality gates

3.2 Technology Stack at a Glance

Layer	Technology	Purpose
Crawling	Scrapy, curl_cffi, httpx	Listing and menu extraction
Anti-blocking	Oxylabs residential proxies	UAE-exit IP rotation
Anti-fingerprinting	curl_cffi (Chrome impersonation)	TLS-layer bypass
Concurrency	asyncio, ThreadPoolExecutor	Throughput at scale
Classification	MyRA AI (in-house)	std_term and ingredient generation
Machine learning	scikit-learn (TF-IDF, kNN)	Mapping items to known terms
Geo	SciPy (cKDTree, BallTree)	Coordinate-based resolution
LLM	OpenAI GPT-4.1-mini	Restaurant classification, translation
Search APIs	Serper, Tavily	Google Maps data, web research
Marketplace scraping	Apify	Google Maps at scale
Data processing	pandas, ijson, orjson	Large-file streaming
Storage	PostgreSQL, JSON/JSONL	Persistence and delivery
Text	ftfy, unicodedata	Encoding repair, normalisation
Logging	loguru	Structured operational logs

3.3 Codebase

Measure	Value
Python modules	233
Lines of code	52,860
Functional areas	15
Automated test modules	20+

4. Stage 1 — Target Discovery and URL Collection

4.1 The Problem

Talabat does not publish a restaurant directory. Listings are only reachable by browsing a **delivery zone** — a geographic area Talabat uses to decide which restaurants serve a given address. To find every restaurant in the UAE, we must first enumerate every delivery zone, then paginate through each one's listings.

4.2 Zone Enumeration and Coverage

The UAE is divided into **648 Talabat delivery zones** spanning all seven emirates. Our initial national sweep covered **646 of them** — effectively the complete country.

Delivery zones identified	648	
Delivery zones swept	646	(99.7%)
Unique restaurant branches found	17,192	

This is the foundation the entire dataset rests on: because the sweep is zone-exhaustive rather than search-based, coverage is not dependent on guessing the right query terms.

4.3 The Extraction Technique

Talabat's website is a Next.js application. Rather than parsing rendered HTML — which is fragile and changes without notice — the pipeline reads the page's embedded application state directly:

```
<script id="__NEXT_DATA__" type="application/json">
```

This single JSON block contains the complete listing data the page will render. Reading it is faster, more reliable, and far less likely to break than HTML scraping. This technique is used consistently across the pipeline.

One critical detail: the `?aid=` URL parameter carries the delivery zone identifier. Without it Talabat serves a generic homepage with no restaurant data. It is load-bearing on every request.

4.4 Pagination Strategy

Zone sizes vary enormously — from a handful of restaurants to over 500. A hybrid strategy handles both:

- **Known page count** (parsed from `totalVendors` and page size): all page requests are issued in parallel.
- **Unknown page count:** pages are chained, each yielding the next, stopping automatically when a page returns empty.

This guarantees no page is missed regardless of zone size, while keeping large zones fast.

4.5 Throughput — Measured, Not Assumed

Residential proxies are latency-bound rather than bandwidth-bound: every request pays 1–3 seconds of proxy round-trip regardless of concurrency. Throughput was therefore benchmarked to find the real ceiling:

Concurrency	Throughput
10	2.9 req/s
25	4.5 req/s
40	5.0 req/s
60	4.6 req/s

Throughput peaks at 40 concurrent requests and *degrades* beyond it. Direct unproxied connections measured no faster (3.5–4.1 req/s), confirming Talabat's own response time is the ceiling — so the proxy costs nothing in speed while providing essential IP rotation.

Production runs at concurrency 40.

4.6 Incremental Discovery

Once the national baseline exists, re-sweeping all 646 zones monthly would be wasteful — most zones return restaurants we already hold. Instead, each zone is **probed** (one request for page 1) to measure its *new-listing rate*, and zones are then ranked by yield:

Zones probed for yield	572
Highest-yield zones crawled	83
Pages fetched	6,447
New restaurant branches found	18,009
Runtime	2.4 hours

This is a deliberate efficiency decision. Peripheral zones return 100% new listings in ~21 pages; saturated central-Dubai zones return 8–20% new across 400+ pages. Crawling by measured yield rather than alphabetically captures the best data first.

An important negative result: an earlier attempt to rank zones by *predicted* efficiency (using vendor density and composition) was measured against actual results and found to have no predictive power. It was discarded in favour of live probing. Restaurant-format composition was similarly tested as a ranking signal and rejected — it is ~44%/32%/13% in every zone measured, so it cannot discriminate between them.

4.7 Operational Design

Every long-running crawl in this pipeline shares the same operational features:

- **Checkpointing** — safe to interrupt and resume; progress survives restarts
- **Failed-page capture** — every failure is written to a retry file so a second pass targets exactly those pages
- **Live status** — a read-only status command reports progress from the checkpoint at any time, from any terminal
- **Append-mode output** — no data loss on crash

5. Stage 2 — Restaurant Identification

5.1 Purpose

A Talabat listing URL is not necessarily a restaurant. The platform also lists grocery stores, pharmacies, florists and pet shops. Before investing in a full menu scrape, each URL is verified.

5.2 Method

Each page carries a `schema.org` JSON-LD block:

```
<script type="application/ld+json">
```

The pipeline reads `@type` and `servesCuisine` to confirm the listing is a food business, and captures the canonical name and coordinates at the same time — independent second opinions on identity and position that are used later during deduplication and change detection.

5.3 Technology

curl_cffi with Chrome impersonation. Standard Python HTTP libraries present a TLS fingerprint that identifies them as automated tools; requests are refused before any content is served. `curl_cffi` reproduces a real Chrome TLS handshake, which is what makes reliable access possible at all.

Ten concurrent workers, branch-ID checkpointing for resumability, and separate output streams for confirmed restaurants, non-restaurants and failures.

6. Stage 3 — Cuisine and Restaurant Classification

6.1 What Is Produced

Three commercial attributes per restaurant, none of which Talabat publishes:

Field	Values
restaurant_type	Full-Service Restaurants, Quick-Service Restaurants, Cafes, Bakery, Cloud Kitchen
outlet_type	Chain, Independent
chained_outlet_type	Local Chain, MNC Chain, N/A

6.2 A Four-Stage Design

Classification is deliberately staged so that expensive methods only run where cheap ones cannot answer.

Stage 1 — Free local signals

Computed from data already held, with no API calls and no cost:

- **Menu profile.** We hold every restaurant's complete standardised menu. What a business actually sells is strong evidence of its format — a menu that is 80% beverages and cake is a Cafe; one dominated by combos and burgers is Quick-Service.
- **Prior classifications.** Restaurants already classified in earlier months are reused directly when the brand name matches.

These produce hints, not verdicts.

Stage 2 — Google Maps footprint via Serper

One query per distinct brand returns the count of same-brand UAE locations, which directly answers Chain vs Independent, plus coordinates, Google Maps CID, category and rating.

Three filters are applied, each because the raw count is wrong without it:

1. **Title matching.** Google returns fuzzy matches — a search for "Sour Mango" returned "Sour Bliss" and "Mister Mango". Counting raw results would classify single-outlet cafes as chains. 2. **Arabic titles.** Fuzzy name comparison scores an Arabic-script title against its English equivalent at near zero. Arabic is stripped before comparison so a brand is not counted as two different businesses. 3. **Non-UAE results.** The country parameter biases results but does not restrict them; matches in Bahrain and Oman are dropped.

Stage 3 — Tavily web research, fallback only

Serper answers "how many UAE locations". It cannot answer whether a chain is **local** or **multinational** — that requires narrative evidence ("founded in Lebanon, 40 outlets across the GCC"). Tavily is fired only where Serper leaves a genuine gap, keeping cost proportionate.

An API key pool rotates automatically on rate-limit responses.

Stage 4 — LLM decision

GPT-4.1-mini receives the rule hints and the Google Maps evidence together and returns the final classification, batched 30 per request for efficiency.

Two design decisions matter:

- **Every value is validated against allowed sets** and coerced. An unrecognised answer falls back to the rule hint rather than shipping.
- **Google Maps evidence overrides the model.** Where the model's inference disagrees with what Maps actually shows, the evidence wins — the client's standard is the real-world footprint, not a model's guess from a name.
- **`chained\_outlet\_type` is derived, never model-generated.** Independent always yields "N/A"; only Chains are asked Local vs MNC. Allowing free-form output previously produced whitespace variants that polluted the field.

7. Stage 4 — Menu Extraction

7.1 Scale

The largest data-volume stage in the pipeline: every restaurant's complete menu, re-captured every month.

Restaurants scraped per cycle	22,327
Menu items captured	2,094,478
Average items per restaurant	94

7.2 The Hidden-API Technique

As with listings, menus are read from the page's embedded application state rather than rendered HTML:

```
props.pageProps.initialMenuState.menuData.items[]
```

Seven fields are captured per item, each chosen deliberately:

Field	Source	Note
branch_id	URL path	The restaurant identifier
item_id	Item id	Talabat's server-side identifier
item_name	Item name	As the restaurant wrote it
item_key	Normalised name	Lowercased, single-spaced — the join key
category	originalSection	The section as the restaurant named it, not a display variant
price_aed	price	Already AED in Talabat's UAE SSR
description	Item description	Often absent

7.3 Infrastructure

Component	Detail
Proxy	Oxylabs residential, UAE exit nodes
Client	curl_cffi with Chrome impersonation set on the session
Rate limiting	Global limiter, 2.0 req/s sustained
Resilience	Circuit breaker halts the run on sustained failure
Recovery	Per-restaurant checkpointing; failed restaurants retried separately

7.4 Delisted Restaurants

Restaurants that disappear between months are not errors — they are business closures or platform delistings, and they are commercially meaningful. They are recorded with status and reason, and their previous menu data is retained rather than deleted, so historical analysis remains intact.

8. Stage 5 — Menu Standardisation and Enrichment

This is the largest engineering investment in the pipeline and the stage that creates the most analytical value.

8.1 The Problem

2.1 million menu items written by 22,327 different restaurants, in two languages, with no shared vocabulary. The same product appears as:

```
chicken shawarma • shawarma sandwich • chicken shawarma sandwich
shawarma sandwich • شاورما دجاج
```

Without standardisation, no cross-restaurant analysis is possible. A client cannot ask "what is the average price of chicken shawarma in Dubai" because the data does not know those are the same product.

8.2 MyRA AI — Our In-House Classification Engine

The standardised vocabulary is generated by **MyRA AI**, our in-house AI application. MyRA AI performs the actual classification work: given a menu item — its name, category and description — it determines the standardised product term (`std_term`) and derives the constituent ingredients.

How MyRA AI is used:

- **Initial build (June–July).** The entire menu catalogue was classified from scratch through MyRA AI, producing the reference dataset of **355,726 classified menu items** with standardised terms and ingredients. This became the authoritative vocabulary the pipeline maps against.
- **Ongoing months (August onward).** New restaurants bring new menu items. Items that already exist in our classified reference are mapped automatically. Items that are genuinely new — never seen before — are sent to MyRA AI for classification, and the results are returned as Excel for review and integration.

This is the core operating model: **reuse what MyRA AI has already classified, send only genuinely new items for fresh classification.** It keeps classification cost proportionate to genuine novelty rather than to dataset size, and it guarantees that an unchanged menu item carries the same term this month as last.

For the August cycle, MyRA AI classified **86,997 new item keys**, returned with finalised standard terms, taxonomy and ingredients.

8.3 The Mapping Tiers

Every menu item is resolved through an ordered set of tiers, from most authoritative to least:

Tier	Source	Coverage (August cycle)
1	MyRA AI reviewed output — authoritative	874 keys
2	Previous month's shipped export — continuity	310,061 keys / 93.0% of rows
3	MyRA AI reference, exact match	10,026 keys
4	TF-IDF kNN against the reference	41,967 keys

Tier 2 exists for a commercial reason, not a technical one. An unchanged menu item must not silently acquire a different `std_term` this month than last — otherwise every price-comparison report would show phantom changes. Reusing last month's label guarantees month-over-month stability.

8.4 The Matching Engine

For items not resolved by exact match, a text-similarity model maps them onto the nearest known classification.

Method: TF-IDF vectorisation combining character n-grams (2–3, within word boundaries) and word n-grams (1–2), compared by cosine similarity, with a 5-nearest-neighbour majority vote.

Why character n-grams: they make the match robust to how menus are actually written. shawarma, shawrma and shwarma share nearly all their character sequences, so they land in the same neighbourhood without a spell-checker or a synonym list.

Why the category is included: the item's menu category is fused into the matched text. This is worth **+18 accuracy points** — chicken alone is ambiguous across sandwiches, curries, grills and soups; chicken || Sandwiches is not.

Confidence signal. How many of the five neighbours agree predicts correctness far better than the similarity score does:

Neighbour agreement	Accuracy
5 of 5	97.5%
4 of 5	84.0%
3 of 5	64.7%
2 of 5	38.0%

Items below the acceptance threshold are flagged for review rather than shipped as fact. The accepted set runs at **93.8% accuracy**, and the flagged remainder concentrates roughly 85% of all errors into 43% of the rows — which is what makes targeted human review affordable.

8.5 Method Selection — Measured, Not Assumed

Before committing to this approach, alternatives were measured on identical held-out data:

Method	Accuracy
TF-IDF kNN, name + category	76.5%
TF-IDF kNN, name only	58.6%
With descriptions added	74.2%
OpenAI text-embedding-3-small	62.3%
HuggingFace multilingual-e5-small	56.8%
Direct LLM classification	42.5%

The character-based method outperforms both semantic embeddings and direct LLM classification, at zero API cost and roughly 100× the speed. Menu names are short keyword strings rather than sentences; embeddings compress them into meaning, and meaning is what misleads — the task is "which label was assigned to items like this", not "what is this food".

These measurements are recorded so the same ground is not re-tested each month.

8.6 Text Sanitisation

All shipped text passes a deterministic cleaning pipeline:

Step	Purpose
Unicode NFKC normalisation	Fold full-width and ligature forms
Control and zero-width removal	Invisible characters break grouping silently
Mojibake repair (ftfy)	Fix double-encoded text
Emoji removal	Talabat decorates section names
Arabic translation	Per requirement, shipped data carries no Arabic
Whitespace collapse	Copy-paste artifacts
Smart title-casing	Consistent presentation, acronyms preserved

Arabic text is **translated, not discarded** — brand names are transliterated (بيبيسي → Pepsi) so no product is lost to a formatting rule. Every change is written to a reversible mapping file.

8.7 City Resolution

Talabat does not reliably provide a city. Where the address is absent, city is derived from coordinates using a k-nearest-neighbour model trained only on records whose address and coordinates come from the same verified Google listing.

Method	Accuracy
Coordinate kNN, verified labels	99.45%
kNN on brand-level labels	72–81%
kNN on parsed-address labels	69–79%
Geometric rule	62%
LLM given coordinates	47.2%

The determining factor was **label provenance**, not algorithm choice — the same model moves from 69% to 99.45% purely by training on labels that genuinely belong to their coordinates. A distance guard leaves the city null rather than guessing when no nearby labelled point exists.

9. Stage 6 — Business Data Enrichment via Google Maps

9.1 The Gap

Talabat publishes a restaurant's delivery zone, but not its street address, phone number, website, precise coordinates or Google Maps presence. For a commercial dataset these are essential — they are what make a listing a real, locatable business rather than an app entry.

This stage recovers them from Google Maps.

9.2 Approach — Apify for Scale

Google Maps is heavily defended and expensive to scrape directly. An in-house Playwright scraper with residential proxies was built and tested, and abandoned on measurement: Google Maps' own JavaScript bundle pulls ~4.5 MB per page load, making bandwidth costs worse than paying Apify's per-result fee.

The production approach uses the **Apify `compass/crawler-google-places` actor**, which returns structured Google Maps listings at a predictable per-result cost.

Query format. A bare restaurant name often returned nothing useful — too short or too generic. Appending a fixed suffix fixed this reliably:

```
"{Restaurant Name} UAE Restaurant"
```

This became the standard query format used across the project.

Match validation. Early runs matched wrong businesses — "Bao Kitchen" matched "Dragon Bao Bao Restaurant". This was fixed with an **ordered-prefix token match**: stopwords are stripped from both names, and the Google listing must literally begin with the Talabat name's tokens in order. This matching algorithm is reused throughout the project.

A **46-brand allowlist** of known major chains received a higher result cap (150 vs 30), since large chains genuinely have many locations to find.

9.3 Coverage Achieved

Enrichment ran across all **15,768 Talabat branches**:

Outcome	Branches	Share
Matched — real address / phone / website found	11,116	70.5%
— via Apify / Google Maps	7,101	
— via Tavily web-search recovery	4,015	
Chain confirmed, specific branch not determinable	171	1.1%
No public footprint found	3,827	24.3%
Not reached	635	4.0%

Among the 11,116 matched branches:

Address	99.4%
Phone	81.0%
Website	68.2%

The unmatched remainder are overwhelmingly small informal operations — home kitchens and single stalls — that have no public web presence of any kind.

9.4 Two-Tier Enrichment Strategy

Prominent chains and MNC brands were enriched brand-by-brand through dedicated Apify runs, each brand producing its complete UAE footprint. Over 25 major brands were processed individually — McDonald's, Starbucks, Subway, KFC, Burger King, Costa Coffee, Dunkin', Tim Hortons, Pizza Hut, Popeyes, Shake Shack, Caribou Coffee, Peet's Coffee, Jollibee, Five Guys, Pret A Manger, Caffè Nero, Zaatar W Zeit, Albaik, Arabica and others.

Independent and less prominent restaurants — the long tail, where a brand-level search returns nothing useful — were enriched through the **Serper API**, querying Google Maps per restaurant. Serper's `/maps` endpoint returns the full Maps listing (19 fields including address, phone, website, place ID, opening hours) rather than the search card, at the same credit cost.

Web-search recovery. For restaurants Google Maps had no listing for, a third pipeline used Tavily web search with LLM extraction to find any structured presence — an official website, a Zomato or Deliveroo listing — recovering a further 4,015 branches.

9.5 Two Valuable By-Products

Hidden chains discovered. The enrichment revealed **49+ Talabat listings that are genuine multi-location chains** operating more UAE branches than Talabat's own listing shows. This was a core commercial insight the work was designed to surface.

Non-Talabat business directory. The searches turned up **18,806 real UAE food-service businesses that are not on Talabat at all** — a ready-made market-expansion and prospecting list, captured at no additional cost.

9.6 Cost Discipline

Apify billing is sensitive to configuration, and cost controls are applied deliberately:

- **No actor-side filters.** The actor bills a per-place surcharge for each filter applied. All filtering is done locally instead.
- **Reviews, images and questions disabled.** Reviews are the expensive component and are not needed.
- **Immediate dataset download.** Free-tier datasets are deleted after 7 days; a completed run whose dataset expired is spend with nothing to show for it.
- **Measure before committing.** Before authorising a full run, a 50-record probe measures what the data actually contains, so a decision to spend is based on evidence rather than assumption.

10. Stage 7 — Logo and Image Collection

10.1 Purpose

Restaurant logos are required for the client application's user interface.

10.2 Two-Phase Design

The work is split deliberately across two scripts, because the two halves have opposite infrastructure requirements:

Phase 1 — URL extraction. Reads each restaurant page's embedded state to find the logo URL. This requires the Oxylabs proxy (Talabat is protected) and runs at a deliberate 1–2.5 second delay per request.

Phase 2 — File download. Downloads the actual image files. These are served from a plain CDN with no bot protection, measured at **55.6 images/second with 20 threads and no proxy at all**.

Combining them would force the fast half to run at the slow half's pace.

10.3 Operational Features

- **Cohort isolation** — each month's logos are separated into their own folder so a handover contains exactly that month's assets
- **Retry capability** — failed downloads are recorded and retried separately rather than lost
- **Worker tuning** — 30 concurrent workers, established by measurement after higher concurrency caused connection failures on Windows
- **Structured logging** via loguru
- **Live status** reporting for both phases
- **Reason capture** — restaurants with no logo record **why**, rather than simply being absent

10.4 Current Holdings

Restaurant logos held	22,322
-----------------------	--------

11. Stage 8 — Monthly Refresh and Change Tracking

11.1 Purpose

The commercial value of this dataset is not a single snapshot — it is the ability to track how the UAE restaurant market changes month to month. Price movements, new products, discontinued items and closures are the analytical product.

11.2 The Refresh Cycle

Each month, every known restaurant is re-scraped and its current menu compared against the previous month's stored baseline.

11.3 Change Detection

Two layers of comparison:

Restaurant level — a hash of the complete menu detects whether anything changed at all, allowing unchanged restaurants to be skipped cheaply. The hash is computed over a **sorted** representation so that item reordering is not mistaken for a change.

Item level — for restaurants that did change, each item is compared field by field:

Change type	Meaning
new_item	Product added to the menu
removed_item	Product discontinued
price_change	Price moved — with old value, new value and delta
description_change	Product description revised
category_change	Item moved between menu sections

A critical normalisation: the stored baseline has been cleaned and normalised, while a fresh scrape is raw. Comparing them naively reports every item as "changed" purely from formatting. Text is therefore normalised on both sides before comparison, while the raw value is what gets stored.

11.4 Price Velocity

Month-over-month price comparison is a dedicated deliverable for the Technology team's analytics module. Because `source_id`, `item_key`, price, description and category are all captured directly from Talabat, price movements are measured against real platform data rather than derived estimates.

A representative cycle measured **105,289 price changes at a median increase of +7.1%** — independently corroborated by two separate derivations agreeing within 1.2%.

11.5 New Listing Detection

In parallel, newly-listed restaurants are discovered and compared against the existing universe. The comparison distinguishes genuinely new businesses from **re-listings** — where Talabat re-registers an existing venue under a new branch identifier after an ownership change or re-onboarding.

A cascading match ladder runs strongest-rule-first, with the winning rule recorded for every match so results are auditable by strategy rather than resting on a single opaque score.

12. Stage 9 — Deliverable Assembly and Quality Assurance

12.1 Assembly

The final stage compiles every cohort into the single monthly file. It is deliberately **deterministic**: every judgement call is made in an earlier stage and frozen into a configuration file, so assembly itself makes no API calls, fits no models, and produces byte-identical output from identical inputs.

That property is what makes each month's file directly comparable to the last.

Streaming throughout. The pipeline routinely handles files of 300–700 MB on hardware with under 1 GB of spare memory. Reading uses `ijson` (incremental parsing), writing uses `orjson`. A full assembly of 25,455 records and 2.1 million menu items completes in approximately **one minute**.

Field order is preserved so that a month-to-month file comparison shows real changes only.

12.2 Validation Gates

Nothing is released without passing an automated gate that reads the **finished output file** — not the inputs, because the point is to inspect exactly what the Technology team will receive.

Category	Checks
Structural	Every line parses; no duplicate records; no empty menus
Schema	Every required field present, of the correct type
Menu integrity	No missing names, prices or standard terms
Consistency	Identical item names carry identical terms and ingredients
Identity	Brand identifiers never split across a brand
Text	No emoji, mojibake, Arabic, CJK, control or invisible characters
Reconciliation	Record and item counts match the build report exactly

A failing gate blocks release. The build writes to a temporary file and only promotes it after the counters reconcile, so a rejected build never leaves a partial file behind.

12.3 Duplicate Prevention

Duplicate detection is defined by **location, not name**. A brand name repeating is not a duplicate — a chain legitimately has many branches. A brand repeating **at the same place** is.

Four keys are checked, from strictest to loosest:

Key	Catches
Name + coordinates	The same outlet listed twice
Name + address	One outlet whose coordinates differ between sources
Name + area + city	Same-neighbourhood duplicates, reported separately

Coordinates are rounded to approximately 11 metres before comparison — two genuinely distinct branches are never that close, while one outlet's coordinates from two different sources routinely differ by a few metres.

12.4 Test Suites

Every production stage has an automated test suite run before any live run, structured in three layers:

Layer	Purpose
Smoke	Does everything import, load, and produce correct shapes
Sanity	Does the logic behave correctly on hand-built cases
Regression	Do previously-fixed issues stay fixed

Regression cases are not hypothetical — each records a real behaviour observed during development, with its expected value pinned in the test.

Proxy connectivity is verified before every production run.

13. Technology and API Stack

13.1 In-House AI

System	Role
MyRA AI	Menu item classification — generates standardised product terms and derives ingredients. The source of the classification vocabulary the entire dataset is built on.

13.2 External APIs and Services

Service	Used For	Notes
Oxylabs	Residential proxies, UAE exit nodes	Essential for all Talabat access
Apify	Google Maps scraping at scale	compass/crawler-google-places; per-result billing with local filtering to control cost
Serper	Google Maps data per restaurant	/maps endpoint — 19 fields per listing
Tavily	Web research and recovery	Key pool with automatic rotation
OpenAI	Classification, translation	GPT-4.1-mini, batched

13.3 Python Stack

Category	Libraries
Crawling	Scrapy, curl_cffi, httpx, requests
Async	asyncio, aiohttp
Parsing	BeautifulSoup, ijson, orjson, lxml
Data	pandas, numpy
ML	scikit-learn (TF-IDF, NearestNeighbors), SciPy (cKDTree, BallTree)
Text	ftfy, unicodedata, regex
Database	psycopg2, asyncpg, SQLAlchemy
Excel	openpyxl
Logging	loguru
Testing	pytest

13.4 Storage

Store	Contents
PostgreSQL	Restaurants, menu items, ingredients, deltas
JSON / JSONL	Deliverables and pipeline intermediates
Local filesystem	Logo and image assets

14. The Monthly Operating Cycle

14.1 Run Order

#	Stage	Runtime
1	Probe zones and rank by yield	~15 min
2	Crawl high-yield zones for new listings	~2.5 hrs
3	Identify and verify new restaurants	~1 hr
4	Classify new restaurants	~1 hr
5	Re-scrape all known restaurant menus	~8 hrs
6	Compute month-over-month delta	~20 min
7	Sanitise and translate text	~1 hr
8	Map standard terms and ingredients	~1 hr
9	Send genuinely new items to MyRA AI	Review cycle
10	Enrich new listings via Google Maps	~1 hr
11	Collect logos for new restaurants	~2 hrs
12	Build unified deliverable	~2 min
13	Run validation gate	~5 min
14	Release to Technology team	—

14.2 Design Principles

Every stage in this pipeline follows the same operational contract:

- **Dry-run first.** Every script supports a mode that reports what it would do, including cost estimates, without spending or writing.
- **Resumable.** Checkpointing means an interrupted run resumes rather than restarting.
- **Non-destructive.** Raw captures are never mutated; each stage writes new files.
- **Auditable.** Every transformation writes a mapping of what changed.
- **Observable.** Read-only status commands report progress at any time.
- **Validated.** Nothing ships without passing an automated gate.

14.3 Scaling to New Cohorts

Adding a new month is a single configuration entry. The assembly pipeline is cohort-driven by design, so September's data joins July's and August's without code changes.

The same architecture extends to additional platforms — Zomato UAE, Deliveroo, Noon Food, HungerStation — as the extraction and enrichment stages are already platform-agnostic below the fetch layer.

15. Data Model

15.1 Core Tables

Table	Contents
talabat_restaurants	One row per branch: identity, location, classification, enrichment
talabat_menu_items	One row per menu item per branch, with standard term
menu_item_ingredients	One row per item-ingredient pair
ingredients_taxonomy	816 distinct ingredients with categories
mnc_chain_locations	Complete UAE footprint per major chain brand
talabat_menu_deltas	Month-over-month change log

15.2 Key Identifiers

Identifier	Definition
branch_id / source_id	Talabat's own branch identifier — the primary join key
restaurant_id	Talabat's chain identifier
chain_id	Our stable brand identifier, derived deterministically from the normalised brand name
item_key	Normalised menu item name — the menu-level join key

chain_id is a **pure function of the brand name**, verified by reproducing 4,000 of 4,000 shipped identifiers from names alone. A brand appearing in two different months therefore lands on the identical identifier with no lookup table and no reconciliation step.

16. Appendix — Codebase Map

Directory	Purpose	Modules
url_collector/	Listing URL discovery	6
listing_scraper/	Zone crawling, prioritisation, yield measurement	14
Restaurant Identifier/	Restaurant verification	2
tavily/	Cuisine and type classification	24
August_classification/	Serper + Tavily + LLM classification	8
talabat_2_0/, TAB_MENU_VALIDATION/	Menu extraction and validation	20
menu/	Menu cleaning, standard-term mapping	21
August_menu/	Monthly cohort build	18
menu_refresh/	Refresh, delta, enrichment	28
map_version2/	Google Maps enrichment, chain footprints	6
apify/	Apify orchestration and processing	16
listing_comparison/	Month-over-month listing comparison	9
export/	Deliverable generation, location cleanup	27
images/	Logo and image pipeline	8
unified/	Unified deliverable assembly	5
Postgre/, supabase/	Database migration and loading	6
ingredients/, sanitization/	Ingredient loading, text cleaning	5
tests/	Cross-cutting test suites	5

Total: 233 modules, 52,860 lines of Python.

End of document.